

Assessment 수집 · 평가 시스템

2팀 · 2026. 07

Contents



1. 프로젝트 개요
2. 시스템 구성도
3. 준비 사항 및 필수 환경
4. 에이전트 기능 상세
5. 엔진 기능 상세
6. 자원 평가 및 산출물
7. 활용 (범용)
8. 테스트 환경·재현
9. 시나리오 정의
10. 향후 발전

프로젝트 개요

주어진 환경의 자원 적정성 평가가 주된 목표

배경 · 문제

- 재해복구/마이그레이션 대상의 OS·자원 스펙 파악 필요
- 수동 파악의 한계 — 누락·오류 다발
- 파악값을 그대로 재해복구/마이그레이션에 적용 시 오버/언더 프로비저닝 위험

목표 · 산출물

- 주된 목표: 주어진 환경의 자원 적정성 평가 (범용)
- 주 산출물: 환경·각 서버의 자원 적정성 평가 + 고객/엔지니어 보고서
- 부가 산출물: assessment-API(GET /api/assessment)로 재현·사이징을 외부 자동화가 소비하도록 노출 (프로비저닝 근거)

배경: 자원 적정성 평가의 필요성

수동 파악의 한계와 오판의 비용 — 실측 기반 판정의 근거

수동 파악의 한계

문제	결과
OS · 자원 스펙을 사람이 조사	서버 많을수록 누락 · 오기입 — 전수 점검 불가
명목 스펙(카탈로그 값)만 봄	실제 부하를 모름 — 과다 할당도 병목도 안 보임
일회성 스냅샷	추세(용량 소진 · 상승) · 포화를 못 잡음

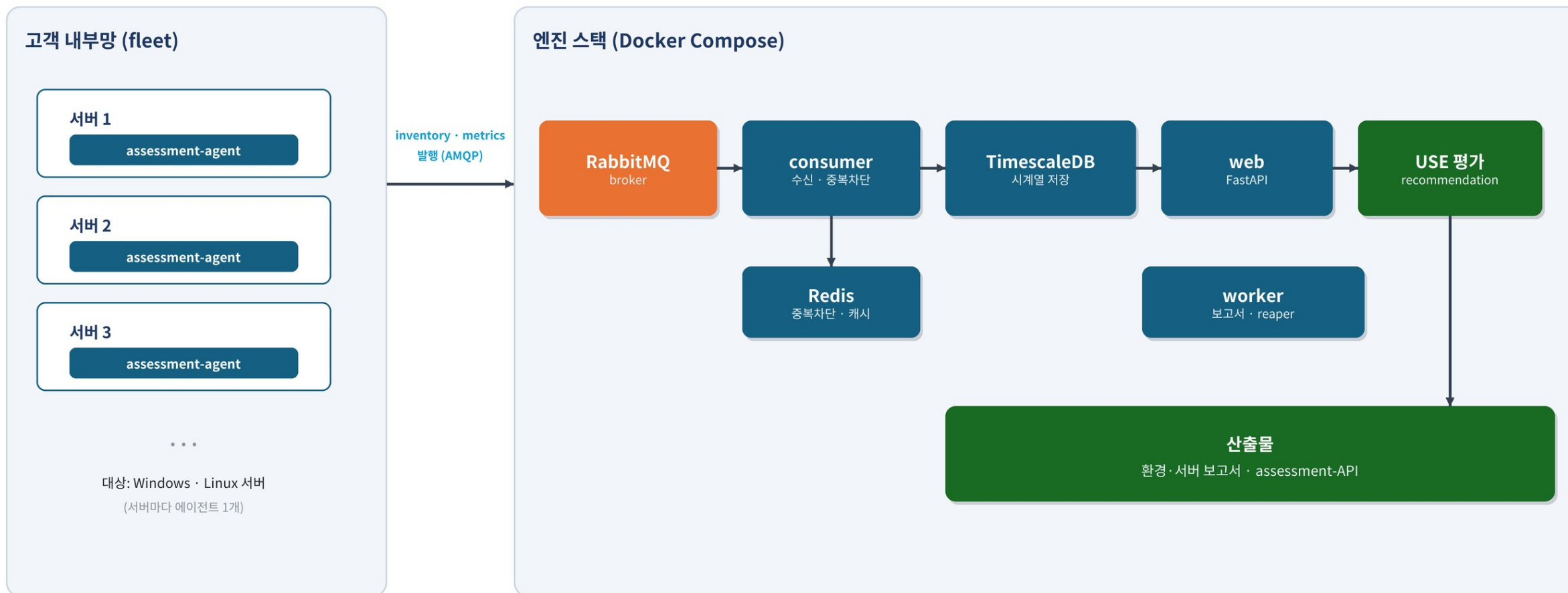
오판의 비용 — 안전 우선의 근거

오판 방향	대상	비용
under-provision (과소)	대상이 부하를 못 받음	재해복구 · 이전 후 성능 장애 · 중단 (비쌈)
over-provision (과다)	안 쓰는 자원에 지불	지속 비용 낭비 (덜 비싸도 항상 씬)

- 실측 부하 기반 판정으로 두 오판을 동시에 교정 — 우선순위는 안전(under 회피) > 정확 > 절감

시스템 구성도

고객 내부망 fleet → 에이전트 발행 → 엔진 스택(수신·저장·평가) → 산출물 (제품 자체로 완결)



아키텍처 개관: 핵심 설계 결정

구성 요소 분리 기준 — 각 결정의 근거

결정	내용	근거
에이전트 / 엔진 분리	에이전트는 raw 수집 · 발행만, 계산 · 판정은 엔진 한 곳	관측과 계산 분리 — 재시작 · 버전차 드리프트 제거, 에이전트 stateless로 견고
단일 바이너리 2전략	OS 계열당 바이너리 1종 (Linux musl static · Windows i686 dispatch)	OS 세대 폭발을 링크 · 로드 단계에서 흡수 — 런타임 분기 최소화
MQ(RabbitMQ) 경유	발행을 브로커가 완충 · 큐 상한 · DLQ 격리	발행 폭주 방어 · 비동기 · 소비 측 단기 장애 회복 창
TimescaleDB 시계열	hypertable(시계열 자동 분할) + continuous aggregate(사전 집계)	raw 누적 카운터를 미리 계산해 둬 — 장기 조회가 원본 전량 스캔 안 함
규칙 기반 진단	수치는 결정론 규칙 산출 · 임계는 공식 표준 출처	추적 · 감사 가능 · hallucination 없음 — 안전 최우선과 정합

- 관통 기준: 자원 적정성 오판은 장애로 직결 → 모든 설계 결정이 추적성 · 안전을 최우선으로 둔다

관통 설계 원칙

에이전트부터 API까지 일관되게 적용한 5가지 기준

원칙	내용	적용 위치
안전 > 정확 > 절감	애매하면 안전한 쪽(더 큰 크기). 다운사이즈는 근거 충분할 때만	사이징 규칙 · assessment-API 안전 불변식
정직한 데이터	값(0 포함)=실측 / null=측정불가를 끝까지 구분 (가짜값 금지)	에이전트 null 의미론 · 엔진 confidence
관측과 계산 분리	에이전트는 raw만, 계산 · 판정은 엔진 한 곳	stateless 에이전트 · 엔진 recommendation
계약 중심 경계	팀 · 컴포넌트 경계를 스키마로 고정 (additive-only 진화)	wire schema_version · assessment-API
규칙 기반 · 추적성	수치는 규칙, 임계는 공식 표준 출처 — 뿌리 없는 값 없음	규칙 기반 진단 · 임계 근거 표

- 이름의 정직함: 'AI 진단' 표현을 스스로 걷어내고 규칙 기반으로 한정 — 명칭이 실재를 앞서면 그 간극이 곧 리스크

준비 사항 및 필수 환경

배포·운영에 필요한 요구사항 · 설치 정보 · 기술 스택

필수 환경 및 요구사항

- 대상 서버: 지원 OS 범위 내, 설치에 root/Administrator 권한 (포트-PID 매핑·서비스 실행경로 수집 위해)
- 엔진 호스트: 같은 내부망, Docker 실행 가능 (VM 또는 물리)
- 네트워크: 에이전트 → 엔진 RabbitMQ 도달

설치 및 구성

- 에이전트: install 서브커맨드 (Linux SSH / Windows WinRM·RDP / rescue offline · Windows offline hive)
- 엔진 스택: Docker Compose — web · consumer · worker · PostgreSQL(TimescaleDB) · Redis · RabbitMQ (배포 스크립트 한 번으로 기동)

사용 기술 스택

- 에이전트: C99 (Linux musl static / Windows mingw i686)
- 엔진: Python 3.12 · FastAPI · aio-pika · SQLAlchemy · TimescaleDB(PG16) · Redis 7 · RabbitMQ 3.13

에이전트: OS 커버리지 & 단일 바이너리

OS 세대에 무관한 단일 바이너리 이식성

구분	지원 범위	이식성 방식
Linux	커널 2.6.32+ (glibc/배포판 무관) EL6~9 · Ubuntu · Debian · SUSE11~15 · Amazon · Oracle	musl 완전 정적 링크 단일 바이너리
Windows	NT5.2(XP SP2/Server 2003) ~ NT10(2016+/Win7~11)	단일 i686 + 런타임 세대 dispatch

- 정적 링크: musl(기본) · rabbitmq-c(MQ 발행) · OpenSSL 1.0.2u(TLS·해시) · libcurl(설치파일 다운로드)
- 런타임 세대 분기: agent_is_nt6로 판별, NT6+ 전용 API는 GetProcAddress로 동적 로드
- NT6+ 전용 API를 import 테이블에 정적 링크하지 않음 → 구세대(XP/2003) 로드 실패 방지

에이전트: 수집 능력·안전

수집 항목 + 읽기 전용·최소 권한 (외부 유틸 없이 직접)

구분	분류	수집 항목
inventory	식별 · OS · HW	hostname · os(id/kernel/arch) · cpu · mem · 부팅(uefi/secure_boot)
	스토리지 · 네트워크 · 서비스	block_devices[] · net_interfaces[] · lvm · services[] · listen_ports[]
metrics	시계열	cpu · memory · disk · network (+ paging/pressure/filesystem/cgroup)

수집 방법 · 안전 — 읽기 전용, 시스템 변경 없음

- 외부 명령(lsblk/df/ss) 없이 procfs/sysfs · NtQuerySystemInformation 직접 파싱 (metrics 60s · inventory 3600s)
- 읽기 전용 관측 — 설정 · 레지스트리 변경 없음. 표준 OS 관리 유틸(ss · tasklist)과 동일 범주. 자기 호스트만 조회
- root/LocalSystem은 타 프로세스 소유 정보(포트-PID 매핑 · 서비스 실행경로) 수집에만 필요

에이전트: 데이터 계약·신뢰성

정직한 데이터(null 의미론) · 관측과 계산 분리(stateless) · 엔진 무관 안전 발행

원칙	내용	근거
null 의미론	값(0 포함)=실측 / null=측정불가	가짜값(0 채움)이 평가를 왜곡하는 것 방지
stateless	raw 누적 카운터만 발행, rate/util 계산은 엔진	재시작 · 버전차 계산 드리프트 제거
인코딩	datapoint-array — 차원을 attr로 분리 (OTel 관례)	메트릭 이름 폭발 없이 차원 확장
device 안정키	커널 이름 대신 불변키(dm/uuid/serial · MAC)	재부팅 · 재배포에도 같은 장치 식별
major 게이트	설치 명령(task.install) major 불일치 시 거부	호환 깨진 특권 명령 차단

발행 신뢰성 — 엔진 상태와 무관하게 안전

- 발행마다 단명 연결(connect → publish → confirm → close) · publisher confirm으로 전달 확인
- 엔진이 없거나 죽어도 지수 backoff 무한 재시도 · 재시도 버퍼 없음(실패분은 흘려보내 메모리 누적 0)

에이전트: 원격 설치 worker (ZDM)

엔진 명령(task.install)으로 ZDM을 원격 설치하는 worker

- 흐름: task.install 수신 → tarball 다운로드 → sha256+크기 검증 → 설치 → task.result 회신

admin 권한 설치 — 안전장치

- schema_version major 게이트 (버전 안 맞으면 거부)
- 다운로드 호스트 화이트리스트 (허용 호스트만)
- 한 번에 하나만 설치 (single-slot)

중복 · 격리 처리

- idempotency: 같은 명령이 중복 도착해도 한 번만 실행 (완료 · 실행중 · 결과 확인)
- fd 격리: 물려받은 소켓 · 파일 핸들(fd)을 실행 전 닫아 설치 프로그램의 접근 차단

계약 버전: schema_version

major . minor (현재 1.0)

minor bump 1.0 → 1.1
필드 추가 · 하위호환 → 수용

major bump 1.0 → 2.0
구조 변경 · 호환 깨짐 → 설치 거부

엔진: 메시지 수신·저장 경로

MQ 경유·2단 idempotency의 근거 — 발행 폭주 방어 · 중복 도착 무해



- exchange 2종(direct · durable): assessment(inventory · metrics · error) · assessment.tasks(설치 명령 · 결과). 큐는 엔진이 선언, 에이전트는 발행만
- 상한 · 격리: metrics 72h · 100만 건(브로커 폭주 방어) · 실패 · 만료 · 상한 초과는 격리 큐(DLQ)로 보관 (유실 대신 운영자 확인)

idempotency 2단 — 같은 메시지가 중복 도착해도 한 번만 저장

단계	방식	장애 시
1단 Redis	메시지 id를 24h 선점 기록 — 가장 빠른 중복 차단	Redis 죽어도 통과 — 2단이 흡수
2단 DB	시계열 자연키에 UNIQUE 제약	중복이면 조용히 무시 (덮어쓰기 없음)

- auto-register: 미등록 agent_id의 inventory 도착 시 서버 자동 등록 · 1 메시지 = 1 트랜잭션(Redis 후처리 실패가 ack 안 막음)

엔진: 저장 계층

TimescaleDB 시계열 + agent_id 단일 키 식별

시계열 저장 — TimescaleDB

- hypertable 5종(수집 시각 기준 자동 분할): 메트릭 · 디스크 I/O · 네트워크 I/O · 마운트 사용량 · 인벤토리 이력
- continuous aggregate(사전 집계, 5분 버킷): CPU 시간 · 디스크/네트워크 바이트 등 raw 누적 카운터를 미리 계산 — 14일 조회가 원본 전량 스캔을 안 함
- 카운터 리셋(재부팅 · 에이전트 재시작 · 값 되돌림) 자동 처리 — 값 감소를 리셋으로 보고 구간을 건너뛴

식별과 저장 — agent_id 단일 키

키	역할	성질
agent_id (UUID)	서버 행 식별·갱신(upsert) · 태스크 라우팅 키	불변 — 에이전트가 첫 실행 시 1회 생성·영구 저장
composite_id · machine_id	복제 충돌 진단용 감사 컬럼	가변 — 식별·라우팅에 미사용
public_id (UUID)	화면·API의 서버 식별자	내부 정수 키를 외부에 노출하지 않음

- 재배포·재부팅으로 MAC이 바뀌거나 hostname을 바꿔도 같은 agent_id면 같은 행 갱신 — 중복 레코드·재연결 로직 없음
- 스키마는 마이그레이션 단일 진실 — 기동 전 전용 컨테이너가 최신 스키마로 자동 반영

엔진: 운영 · 배포 · 명령 이력

검증된 이미지만 배포 · 특권 명령(task.install)은 발행 · 이력을 엔진이 통제

- 배포 단위: 내부망 VM 1대에 Docker Compose 단일 스택(web · consumer · worker + TimescaleDB · Redis · RabbitMQ). outbound-only(VM이 이미지를 받아감)

배포 절차 — 버전 태그로 배포 스크립트 한 번



항목	방식	얻는 것
공급망 검증	이미지 서명(cosign) · SBOM · 빌드 출처(provenance) 자동 첨부	출처 불명 이미지 배포 차단
비밀 관리	파일 기반 시크릿(환경변수 미노출) · 약한 기본값이면 기동 거부	자격증명 유출 · 오배포 방어

특권 명령 통제 — task.install 발행 · 이력 (수신 · 실행은 에이전트 worker)

- 진행 중 1건 제한(중복 설치 차단) · 마감 창 = 큐 만료(좀비 작업 방지) · 무회신은 reaper가 timeout 전이
- 발행 이력은 감사 로그로 보존 — 언제 누가 어느 서버에 설치를 걸었는지 추적

엔진: 규칙 기반 진단

수치는 규칙이 산출 · 임계는 공식 표준 근거 (추적·감사 가능)

- 진단 · 사이징 수치는 결정론적 규칙 엔진이 산출 — 같은 입력이면 항상 같은 판정
- LLM이 수치를 생성하면 추적·감사 불가 + hallucination 위험 → 안전 최우선 원칙과 충돌
- 임계·규칙은 모두 공식 표준(AWS·Azure·USE Method·MS) 출처 추적 — 뿌리 없는 값 없음

임계 근거 — 모든 값이 공식 표준 출처 추적

신호	임계	USE 측	출처
CPU 이용률	p95 >= 70%	Utilization	AWS Compute Optimizer (Balanced)
메모리 이용률	p95 >= 90%	Utilization	Azure Advisor
CPU 포화	run queue/core >= 1.0 (Linux) · >= 2 (Windows)	Saturation	USE Method · Microsoft
디스크 I/O	응답 지연(await) p95 > 20ms	Saturation	VMware · SQL Server 관례
디스크 용량	runway < 30일 · used >= 85%	Utilization	용량 계획 · monitoring 표준

USE Method: 자원 x 3축 평가 매트릭스

4 하드웨어 자원(디스크는 용량·I/O 2측면) x USE 3축 — 각 셀이 판정 신호

- 에이전트가 raw 수집 → 엔진이 각 자원을 USE 3축(Utilization·Saturation·Errors)으로 판정 (14일 창 p95)
- 4 하드웨어 자원(CPU·메모리·디스크·네트워크). 디스크는 용량·I/O 2측면으로 갈려 총 5개 판정 레인

자원	Utilization (이용률)	Saturation (포화)	Errors (오류·고장)	사이징
CPU	사용률 p95 >= 70%	실행큐/코어 >= 1.0 (Win 2.0)	MCE	코어 수
메모리	사용률 p95 >= 90%	사용률 AND 하드폴트 (dual-gate)	OOM · 손상	용량(MiB)
디스크 용량	used% · inode >= 85%	runway < 30일	fs 오류	확장 GB
디스크 I/O	%util (정보용)	await p95 > 20ms	디스크 오류	포화 표시
네트워크	처리량 / 링크속도	drop · conntrack >= 80%	재전송 · NIC 오류	혼잡 플래그

- 사이징 숫자(증설/감축)는 CPU·메모리·디스크 용량 3자원만 — 디스크 I/O는 포화 여부만, 네트워크는 혼잡 플래그만
- Errors는 사이징에 안 섞고 health(고장) 신호로 분리 (창 내 신규 발생 > 0)

자원 적정성 분류 · 근본원인

자원별 판정을 호스트 하나로 종합 (우선순위 순) · 근본원인은 진단 근거

호스트 종합 분류 (host_status, 우선순위 순)

분류	조건
under_provisioned (부족)	자원 하나라도 under · io_bound · filling
idle (유향)	CPU p95 <= 3% AND 네트워크 <= 2Mbps (위 미해당)
over_provisioned (과잉)	CPU 또는 메모리가 over (위 미해당)
insufficient_data (표본 부족)	전 자원 미측정
optimal (적정)	그 외

- under > idle 우선 — 자원 압박(page-out·await·용량 소진·고이용)이 하나라도 있으면 CPU가 낮아도 '미사용'이 아니라 '부족'
- 네트워크 혼잡은 호스트 under로 분류하지 않고 별도 플래그(orthogonal)로 노출 — 사이징 축이 아닌 품질 신호

근본원인 종합 — 진단 근거 (처방은 자원별 독립)



측정 신뢰도 · 처방 규칙

분류와 별개로 측정 불확실성을 4종으로 노출 · 다운사이즈는 근거 충분할 때만

신뢰도 4종 — 분류는 완결하되 불확실성을 종류별로 표기

종류	의미	판정 영향
통계 정밀도	이력 < 30h 또는 부하가 톱 (p95/중앙값 > 2)	분류는 완결 · 신뢰도 하향
커버리지	포화 측 미측정 (Windows perflib 미부착 등)	측정된 측으로 분류 · 미관측 표기
충실도	steal >= 5% — 하이퍼바이저가 가져간 CPU 시간	표본 늘려도 안 줄어드는 편향
정상성	이용률 상승 추세	다운사이즈 등 앞을 내다보는 결정에만

처방(무엇을 몇으로) 규칙

- 목표 이용률 착지: 관측 p95가 목표%에 앞도록 크기 역산 — CPU 목표 70%, 메모리 near-peak 80%
- 증설은 즉시, 다운사이즈는 신뢰도 높음 AND 상승 추세 아님 AND 관측 충분(창 대비 70% 이상)일 때만
- 디스크 용량: 현재 성장률로 365일 버티는 GB. 디스크 I/O·네트워크는 사이징 숫자 없음(티어·혼잡 표시만)

산출물: 고객 · 엔지니어 보고서

환경 · 서버 scope x 고객 · 엔지니어 view — 발행 시점 정적 스냅샷

범위	고객 view (양식 A)	엔지니어 view (양식 B)
환경 (전체 등록 서버)	환경 KPI · 자원 적정성 분포 · 우선 검토 대상	동일 + 정량 지표 · 운영 신호(OS 지원종료 등)
서버 (선택 N대 · 단건)	KPI 8컬럼 요약 — 분류 · 권장 action	정량 16컬럼 — CPU · 메모리 · load · I/O wait · 디스크 · 네트워크 · swap · 재부팅

- 답하는 질문: 우리 환경 자원 배분이 적절한가 / 어디부터 손대야 하는가 / 이 서버는 적절한가
- 발행 시점 정적 스냅샷 — 워커가 비동기 생성 후 보존, 재조회해도 값이 변하지 않음 (이력 추적)
- 발행 흐름: 발행 요청 → 즉시 진행 화면 → 전용 워커가 집계·생성 → 완료 시 본문 표시
- 형태: 웹 화면(HTML) · 브라우저 인쇄로 PDF · 이력 페이지에서 과거 보고서 재조회

부가 기능: assessment-API

외부 자동화가 타겟 VM을 재현·사이징하는 데 필요한 정보를 한 응답으로 (불변 계약)

- 요청: GET /api/assessment — hostname · ip · public_id · pair(동명 호스트 구분)로 조회. 평가 창 window_days(기본 14 · 최소 14)
- 소비자는 내부 식별자를 몰라도 자기 인벤토리가 아는 이름·IP로 조회 가능. 매칭 0건은 오류가 아니라 빈 목록

응답 5블록 — 앞의 4개가 결정용(1급), diagnostics는 선택 소비

블록	내용	소비 용도
identity	public_id · hostname · primary_ip · os_family · online	대상 서버 매칭
reproduction	os · boot · network · storage 트리 · mounts	타겟을 소스와 동일 레이아웃으로 재구성
sizing	축별 current · recommended · action · estimate_quality	타겟 크기 결정
assessment	classification · confidence · data_quality	판정 · 신뢰도 확인
diagnostics	root_cause · resources · advisory	근거 · 감사 (선택)

- 재현 범위 = 레이아웃 청사진: 파티션·LVM/RAID·파일시스템·부트·네트워크 구성을 그대로 복원할 사실을 제공
- 데이터 자체는 API 범위 밖 — 빈 그릇을 소스와 같은 모양으로 만들고, 내용 채우기는 소비자의 복제 채널 몫

assessment-API: 소비 규약 · 안전 불변식

잘못 소비하면 under-provision — 계약이 보장하는 것과 소비자가 지킬 것

소비자가 지킬 것 — 소비 규약

- sizing.axes를 전 원소 순회해 축마다 $\max(\text{current}, \text{recommended})$ 로 프로비저닝
 - 한 축이라도 건너뛰면 그 자원이 통째로 누락 → under-provision. 모르는 축·action이 와도 같은 규약 적용
- contract_version의 major가 다르면 파싱 중단·거부 (의미 드리프트를 조용히 흡수하지 않음). minor 상향은 수용
- 플레이어 매핑은 소비자 몫 — 응답은 플랫폼 중립 원시 수량(vCPU 개수 · MiB · GiB)만 제공

계약이 보장하는 것 — 안전 불변식 (안전 > 정확 > 절감)

불변식	의미
under 축은 recommended $\geq$ current	부족 판정인데 현재보다 작은 값을 주지 않음. 정확 산정 불가면 안전한 하한 + floor 표기
action=keep이면 recommended = current	재계산으로 인한 미세 드리프트 없음 (값 그대로 복사)
disk 축은 축소 없음	increase 또는 keep만 — 온라인 디스크 축소는 파괴적
decrease는 게이트 통과 시만	신뢰도 높음 + 상승 추세 아님 + 관측 충분. 미충족 과다는 keep + diagnostics에만 노출

- 자원 export: POST /api/exports/inventory — 동일 구조 JSON 파일 (Terraform/Ansible 오프라인 투입)

활용: 어세스먼트는 범용 자원 판정 계층

동일 출력(assessment-API·보고서·export)이 여러 맥락에 — 재해복구는 한 응용일 뿐

- 본질: 실측 부하로 자원 적정성 판정(under·over·idle·optimal) + 소스 동일 재현 청사진. 무엇에 쓰이든 흐름·출력 동일

	이전 사이징	운영 최적화	재해복구
목적	계획된 이전 시 타겟 크기 결정	현행 fleet in-place 개선	장애 시 복구 타겟 프로비저닝
타겟 생성	있음 (새 환경)	없음 (현행 수정)	있음 (복구 대상)
주 소비 블록	reproduction + sizing	assessment + sizing + diagnostics	reproduction + sizing
주 소비자	마이그레이션 도구 · IaC	운영자 · 보고서	재해복구 자동화
핵심 가치	실측 사이징 (비용·성능)	비용 절감 · 안정성	안전한 복구 재현

- 세 응용 모두 동일 출력 + 안전 우선 불변식(under 미달 금지 · disk 미축소) 공유
- 어세스먼트는 '무엇에 쓰나'와 독립인 범용 계층 — 이전도·최적화도·복구도 그 판정 위에 얹히는 소비자

활용 1: 클라우드·하이퍼바이저 이전 사이징

기존 스펙을 그대로 옮기는 이전(lift-and-shift)의 과다·과소 답습을 실측 사이징으로 교정

맥락

- lift-and-shift는 과다는 비용, 과소는 성능 부족을 이전 후에도 답습. 수동 사이징은 느리고 실수 잦음

어세스먼트 적용

- reproduction으로 복잡 스토리지(RAID1+LVM+LUKS 등)까지 트리로 재구성 청사진 제공
- sizing.recommended는 실측 부하 기반 — 과다했던 원본을 답습하지 않음. action · estimate_quality로 근거 수준 노출
- 안전 우선이라 이전 타겟이 부하 미달 안 됨(under recommended $\geq$ current · disk 미축소). CPU=p95, 메모리=near-peak 착지
- export(POST /api/exports/inventory)로 동일 구조 JSON을 Terraform · Ansible 등 IaC에 오프라인 투입

가치

- 실측 정확 사이징 = 클라우드 비용 최적 + 성능 보장 동시. 재해복구와 계약(assessment-API) 동일, 맥락만 다름(계획된 이전)

활용 2: 운영 인프라 자원 최적화

이전·복구 없이 현행 fleet을 그 자리에서 딱 맞게 (비용 절감 + 안정성)

맥락

- 감으로 프로비저닝된 fleet에 과다(비용 낭비)·과소(잠재 장애) 공존. 서버 많을수록 수동 점검 불가

어세스먼트 적용

- 환경 보고서로 분포(under · over · idle · optimal) 파악 → '어디부터 손대나' 우선순위
- 과다 · 유휴 → decrease(안전 게이트 통과 시) · 통합으로 비용 ↓ / 과소 → increase로 장애 예방
- disk runway_days · used_pct로 마운트별 소진 잔여일 예측 → 선제 증설 (용량 계획 포함)
- 크기로 안 풀리는 축은 진단으로 — disk_io_tier_hint(티어 상향) · network_congested(혼잡)
- 신뢰도 4종으로 저신뢰 구간은 조치 보류 (잘못된 다운사이즈 방지)

가치

- in-place 최적화 — 마이그레이션·복구 없이 현행 fleet을 딱 맞게. 재해복구와 가장 뚜렷이 구별되는 활용

테스트 환경: OpenStack fleet

다양한 OS 세대에서 에이전트 발행·수집 검증

항목	내용
fleet 규모	Linux 58 + Windows 11 = 69대 (OpenStack)
OS 커버	RHEL·CentOS·Rocky·Alma·Oracle / Ubuntu·Debian / SUSE / Amazon·Tencent + Windows 2003~2025
검증 목적	제각각 OS 세대에서 에이전트 발행·수집 정상 동작 확인

- 사실상 OpenStack Glance의 전 이미지 커버 (UEFI 부팅 결함 일부 제외)
- 전 fleet에 에이전트 배포 후 엔진 MQ로 inventory/metrics 발행 확인
- BIOS/UEFI · systemd/SysV · 32/64비트 등 세대 변수를 한 바이너리로 커버

재현: 선언적 프로비저닝

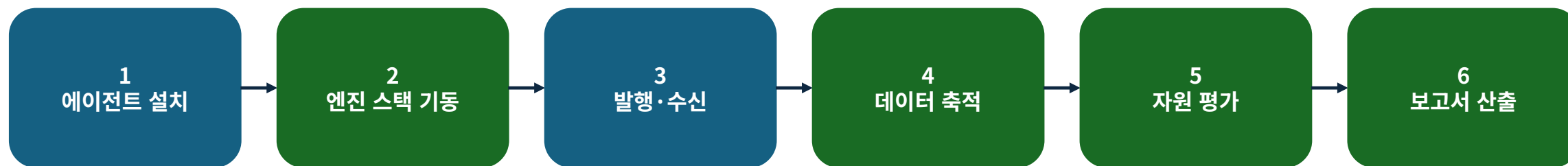
terraform 정의 + provision 스크립트로 재현 가능하게 구성

접근 상황	배포 방법
Linux systemd (표준)	SSH in-place — 바이너리 scp + systemctl restart
cloud-init 부재 (centos6 등)	rescue offline SysV 주입 (네트워크 이후 시작 순서 지정)
SSH 불가 (suse15 등)	rescue offline full 주입 — 바이너리 + env + systemd 유닛
Windows	offline NTFS hive 주입 — exe + 서비스 + env

- terraform으로 fleet VM · 네트워크를 코드로 선언 — 동일 환경 반복 재현
- 재현 절차: 바이너리 빌드 → 인프라 apply → 키 주입 → 에이전트 배포 → 워크로드 · 부하 인가

시나리오: 고객 환경 자원 적정성 평가

에이전트 설치 → 수집 → 평가 → 보고서 (제품 자체로 완결)



파랑 = 에이전트 · 초록 = 엔진 (수집·평가·보고)

- 1. 고객 내부망 대상 서버에 에이전트 설치 2. 같은 내부망에 엔진 스택 기동 (Docker Compose)
- 3. inventory/metrics를 엔진 MQ로 발행 → consumer 수신·저장 4. 일정 시간 데이터 축적
- 5. USE Method로 자원 적정성 평가 · 약점/추천 식별 (엔진의 규칙 기반 판정)
- 6. 환경·각 서버 자원 적정성 평가 보고서 산출 (고객/엔지니어)

향후 발전

설치 진입장벽 ↓ · 규칙 기반의 근거·감사·설명 강화

웹 UI 기반 원격 에이전트 자동 설치

- 엔진 web '서버 추가' 폼에 대상 IP·계정 입력 → 엔진이 원격 접속해 에이전트 설치·기동 → fleet 자동 등록·발행 시작
- 메커니즘 재사용: 에이전트 install 서버커맨드 · 기존 배포 스크립트 · 엔진 worker · 작업 이력 UI
- 부트스트랩이라 task.install(ZDM)보다 한 단계 아래 — MQ가 아닌 직접 SSH/WinRM 도달. 키리스 서버는 볼륨 주입 병행
- 가치: 셸 스크립트 대체로 온보딩 ↑ + 어느 서버가 설치됐는지 중앙 가시성

RAG 기반 임계 보정·감사 (규칙 보완)

- 수치는 여전히 규칙이 산출 — 추적·감사·안전 불변식 유지. RAG는 규칙을 대체하지 않고 근거로 뒷받침
- 벤더 문서(AWS·Azure·VMware·용량계획) 코퍼스로 임계 큐레이션 · 이상값 보정 자문 · 판정 근거 설명(출처 인용)
 - 요지: '수치는 LLM에 안 맡긴다 — RAG로 규칙을 더 잘 만들고 감사·설명.' 규칙 기반 진단 슬라이드와 한 논리
- 제약: 외부 유료 API 금지(로컬 LLM·로컬 벡터스토어). 규칙의 약점(수동 종합·엣지 이상)을 근거 기반으로 보완